

Generation of realistic synthetic cable images to train Deep Learning segmentation models

Pablo Malvido Fresnillo

`pablo.malvidofresnillo@tuni.fi`

Tampere University

Wael M. Mohammed

Tampere University

Saigopal Vasudevan

Tampere University

Jose A. Perez Garcia

Universidade de Vigo

Jose L. Martinez Lastra

Tampere University

Research Article

Keywords: synthetic training data, deformable linear objects, image segmentation, computer vision, Blender

Posted Date: February 23rd, 2024

DOI: <https://doi.org/10.21203/rs.3.rs-3959243/v1>

License: © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

Generation of realistic synthetic cable images to train Deep Learning segmentation models

Pablo Malvido Fresnillo^{1*}, Wael M. Mohammed¹, Saigopal Vasudevan¹,
Jose A. Perez Garcia², Jose L. Martinez Lastra¹

^{1*}FAST-Lab - Future Automation Systems & Technologies Laboratory, Tampere
University, Korkeakoulunkatu, Tampere, 33720, Finland.

²Design Engineering Department, Industrial Engineering School, University of Vigo, Rua
Maxwell, Vigo, 36310, Spain.

*Corresponding author(s). E-mail(s): pablo.malvidofresnillo@tuni.fi;

Contributing authors: wael.mohammed@tuni.fi; saigopal.vasudevan@tuni.fi;
japerez@uvigo.gal; jose.martinezlastra@tuni.fi;

Abstract

One of the main contemporary challenges in robotics is the automation of tasks that involve handling Deformable Linear Objects (DLOs), such as cables or ropes. Due to their changing shape, constant perception is required, which makes computer vision, and in particular, image segmentation, fundamental. Even though image segmentation is a very studied problem, which has been addressed with high accuracy by many different Deep Learning models, they need to be properly trained to segment the objects of interest (DLOs in this case). To do this, it is necessary to have a large and diverse dataset. However, such a dataset hasn't been created for cable images, and doing it manually would be complicated and extremely time-consuming. This paper addresses this issue with a novel methodology, which automatically generates synthetic cable datasets to train image segmentation models. This methodology utilizes Blender to create photo-realistic scenes and a Python pipeline to interact with them. To ensure the diversity of the dataset, before the generation of each image, the pipeline performs random variations on the elements of the scene. After this, the scene is animated, dropping the cables from a certain height and letting them deform naturally after landing, which results in realistic arrangements of the cables. The effectiveness of the methodology was demonstrated by training six popular segmentation models with synthetic datasets and using them to segment real cable images with great results (IoU over 70% and Dice coefficient over 80% for all the models).

Keywords: synthetic training data, deformable linear objects, image segmentation, computer vision, Blender

1 Introduction

Robotics has experienced immense growth in the last three decades and, as a result, robots are nowadays present in almost every industrial field. The majority of their applications are focused

on rigid objects, however, handling deformable objects remains a significant challenge. This additional complexity stems from the need to not only control the object's pose but also its deformation during the manipulation [1]. One particularly intriguing type of deformable objects are

deformable linear objects (DLOs) such as cables, ropes, or sutures. These objects are characterized by having a much larger length than their cross-section dimensions, which brings additional challenges for their robotic manipulation, such as entanglements and larger deformations [2]. These challenges limit the full automation of numerous processes where DLOs are present, such as switchgear cabling [3] or wiring harness assembly in car cockpits [4].

To overcome the manipulation challenges of the DLOs, a proper perception of their shape is required, which can be achieved using computer vision algorithms [5, 6]. The first step for most of the developed algorithms is the semantic segmentation of the DLOs, generally using Deep Learning (DL) models [7]. These models outperform traditional segmentation approaches, such as thresholding or clustering [8], on popular benchmarks [9], however, they need to be properly trained to work correctly. During the robotic manipulation of DLOs, there can be infinite variations in the DLOs, lighting conditions, and background scenes. Consequently, a very large and diverse dataset is required to train these models effectively. Creating such a dataset manually would involve preparing and capturing a vast amount of DLOs images with varying conditions as well as labeling their corresponding ground truth masks, which would be extremely time-consuming. To address this issue, this article presents an innovative methodology for automatically generating realistic synthetic cable datasets for training image segmentation models.

The rest of the article is structured as follows: Section 2 presents a literature review on related research work including deep learning models for segmentation and synthetic data generation. Section 3 presents the approach for creating the synthetic data and illustrates the workflow. Section 4 illustrates the evaluation of the generated data and highlights the performance of the developed approach. Finally, Section 5 concludes the article and presents the possible future work.

2 State of the art

2.1 Deep learning segmentation models

Most DL segmentation models are based on Convolutional Neural Networks (CNNs) [10]. These

networks, which were specifically designed to perform image classification, are constructed as a combination of two building blocks: convolutional and fully connected blocks. Convolutional blocks, composed of convolutional, activation, and pooling layers, extract patterns and features from the image and reduce its spatial resolution retaining the important information. These blocks are normally repeated several times to extract patterns at multiple levels of abstraction. While the first layers detect simple features such as edges, corners, or color gradients; the last convolutions can recognize high-level, and abstract features, like objects or scene context. On the other hand, the fully connected block is the final element of the CNN, and it classifies the image using dense layers and an output layer.

Fully Convolutional Networks (FCN) [11] leverage the hierarchical feature extraction ability of CNNs to perform image segmentation. Typically, they do this using an encoder-decoder structure. The encoder section is composed of a set of convolutional blocks that extract hierarchical features from the input image and reduce its spatial resolution. These blocks are normally taken from well-established CNN architectures, such as VGGNet [12], ResNet [13], or MobileNet [14], which are pre-trained on large datasets such as ImageNet [15]. These CNNs are referred to as the “backbone” of the FCN. On the other hand, the decoder section uses upsampling layers to increase the size of the feature map generated by the encoder and produce a segmentation map of the same size as the input image. Nevertheless, as the final feature map primarily contains semantic information, spatial reconstruction becomes notably challenging. To address this, many FCNs incorporate “skip connections” to merge appearance details from earlier encoder layers with the upsampled feature maps, improving the reconstruction process. This can be seen in Fig. 1.

After the FCN was introduced, its original architecture was extended by many authors, to create new segmentation models with additional features. One of the most popular examples is the U-Net [16], which was initially developed for biomedical image segmentation. This network, as its name says, follows a “U” shape, as its encoder and decoder blocks are symmetric. Thus, each downsampling group of layers in the encoder is connected to its corresponding mirroring group of

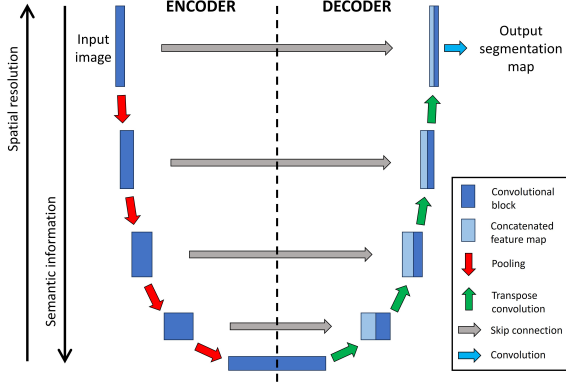


Fig. 1: U-Net schematic representation. Adapted from [16]

layers in the decoder through skip connections. Another example is the LinkNet [17], which is a variant of U-Net that replaces the skip connections with residual connections. These connections make a summation of the down-sampling and up-sampling features instead of concatenating them, which reduces the number of parameters of the network making it more efficient.

Another feature that was incorporated into numerous FCN architectures were pyramid networks, which allow to capture multiscale information (e.g., objects of different sizes). One of the most popular models of this kind is the Feature Pyramid Network (FPN) [18], whose main application is object detection but can easily be adapted to perform image segmentation. This model uses a bottom-up pathway (i.e., a CNN backbone) to extract features from the image and a top-down pathway that, with the aid of skip connections, increases the spatial resolution of the final high-level feature map gradually. Thus, every layer of the top-down pathway can perform object detection at a different scale. This architecture can be extended for image segmentation by applying multilayer perceptrons (MLPs) at each level of the pyramid to generate masks at different scales, which are finally merged.

A different approach for capturing multiscale context is used in Pyramid Scene Parsing Network (PSPNet) [19]. In this model, after a CNN backbone, the resultant feature map is fed into a pyramid pooling module, which pools it at four different scales and processes them with a convolutional layer, to distinguish multiscale patterns.

Then, these pooled features are upsampled and concatenated with the initial feature maps to integrate the multiscale contextual information in the network. Finally, a convolutional layer is used to generate the segmentation mask.

A similar concept, known as Atrous Spatial Pyramid Pooling (ASPP), is used in the DeepLab architectures [20]. However, unlike pyramid pooling, where the feature maps are pooled at different scales, ASPP utilizes parallel atrous convolutions with different dilation rates to capture multi-scale information directly from the feature maps. Atrous convolutions are convolutional operations performed with filters with gaps between their weights, which increase their receptive field without increasing the number of parameters [21]. The spacing between the filter weights is determined by the dilation rate. Therefore, atrous convolutions with different dilation rates can recognize features at different scales. DeepLabv3+ [22] is the most advanced architecture of the DeepLab family. This model concatenates the multiscale features captured using ASPP with the original feature map. Finally, the combined feature map is processed by a convolutional layer to generate the segmentation map.

All the models reviewed in this section have the potential to perform image segmentation with high accuracy and at a fast speed. However, they need to be properly trained for this. Typically, the backbone architectures are pre-trained on large and diverse datasets to learn how to recognize general features and patterns, but the rest of the parameters of the network need to be learned in order to segment the objects of interest. Hence, it is necessary to prepare extensive datasets that consider many different situations, such as different backgrounds, scales, lighting, colors, positions, configurations, or imperfections. However, preparing such a dataset and its corresponding ground truth masks can be extremely time-consuming [23].

2.2 Synthetic image segmentation datasets

As discussed in Section 2.1, the use of DL image segmentation algorithms requires a large amount of training data so they can generalize well and adapt to the variability of real-world scenarios, including lighting conditions, backgrounds, and

object poses. Capturing and labeling all these images manually is an arduous task, hence, numerous researchers have developed solutions to produce them synthetically. One of the most popular techniques is image augmentation [24], which consists of performing multiple transformations (such as rotations, flips, zoom, or changes in color and brightness) to existing images to artificially expand the dataset. One augmentation technique in particular, known as copy-and-paste, has been used by several authors to expand DLOs segmentation datasets [25, 26]. This technique consists of pasting objects (in this case cables), which have been segmented from real images, into different backgrounds, creating new images [27]. These approaches are fast and simple to implement, yet they have several drawbacks, such as requiring some manual work to capture and label the initial images, or not considering variations in features like lighting, shadows, reflections, or the shape of the DLOs, which reduces the realism of the generated images. A similar approach was presented in [28], where synthetic images of aerial power cables are generated by pasting the cables in different background images captured with a drone. However, in this case, the pasted wires are generated using the POV-Ray tracing engine, and therefore, the process can be fully automatic.

The aforementioned approaches constitute valid solutions for simpler applications that don't demand high levels of realism, particularly in scenarios with a small number of DLOs and minor occlusions. However, if the generated dataset is going to be used for training models that will operate in more complex scenarios involving multiple DLOs, varying light conditions, major occlusions, and constant overlapping, these approaches may not be sufficient. This leads to the need for using a more generic material and lighting representation. In this regard, the Physically-based Rendering (PBR) theory aims at producing photo-realistic renders by accurately modeling the light and materials [29]. This theory is the core of several commercial solutions like 3ds Max, Maya, Blender, or Unreal Engine. As an example, the Principled BSDF shader¹ in Blender implements the PBR theory by combining features like Reflection,

Diffusion, Translucency and transparency, Metallicity, Fresnel reflection, and Subsurface scattering among others [30].

As a result, numerous researchers have started to leverage the power of these 3D computer graphics software to generate photo-realistic synthetic images. In particular, Blender stands out as one of the preferred choices among researchers due to its powerful features, versatility, scripting capability, procedural modifiers, and open-source nature. One of the most popular examples is BlenderProc [31], a generic procedural Blender-based pipeline for generating labeled realistic images for training artificial intelligence (AI) models. This solution has been adopted by several authors. Two examples of this are Adam et al., who extended the approach by combining it with domain randomization in [32], and Caporali et al., who employed it to generate synthetic cable images in [33]. However, as BlenderProc is a generic solution, it has some limitations when working with deformable objects. Therefore, in [33] the DLOs had to be modeled as spline curves, reducing slightly the level of realism as the randomness of their shape was not captured. A different Blender-based solution was presented in [34] to generate realistic plant images. In this case, the 3D models of the plants are generated using a commercial software, and then, their mesh is imported into Blender, where the materials and lighting are defined, and the images are rendered. This approach was later extended using generative adversarial networks (GAN) to improve the realism of the synthetic images [35]. Another software that is used for these purposes is the Unreal Engine. An example of this is UnrealCV [36], a tool for generating realistic images for machine learning applications. Finally, to sum up, creating highly realistic images depends on several factors, such as material, lighting, composition, physics, defects, and randomness.

3 Methodology

This paper presents a novel methodology to synthetically generate realistic cable images that can be used to train DL segmentation models. The procedure combines the power of Blender, which allows the creation of photorealistic scenes and the meticulous simulation of cable structures, with a

¹https://docs.blender.org/manual/en/latest/render/shader_nodes/shader/principled.html

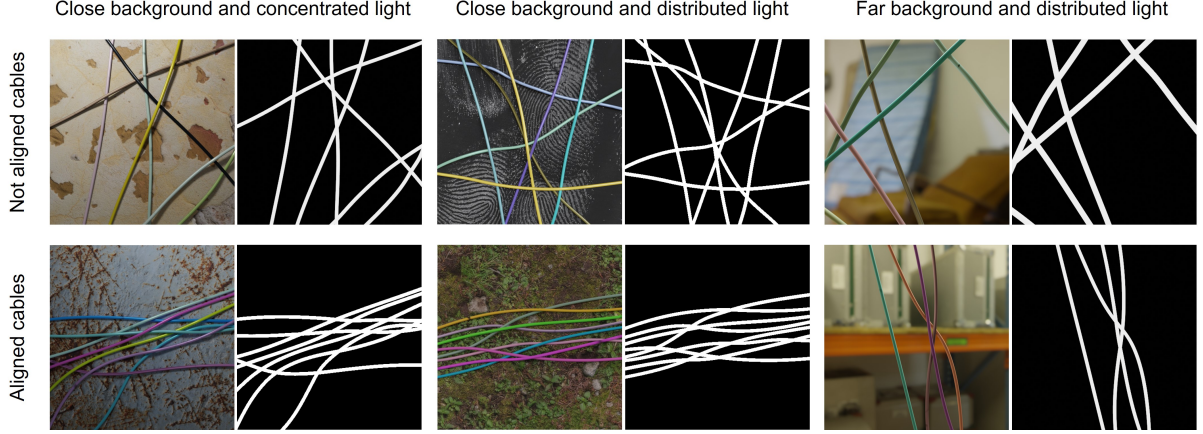


Fig. 2: Synthetically generated cable image categories. More examples of these categories can be found at: https://drive.google.com/drive/folders/1F5uFVP75YcSJReLt_YXrg7I3XGVMhTG9?usp=sharing

Python pipeline that orchestrates the entire process, interacting with the elements in the scene and applying random variations to them. Thanks to this, the system can generate an extremely realistic and diverse cable segmentation dataset in a completely automatic way.

The developed system allows the generation of six different categories of cable images, showcased in Fig. 2. These categories result from the combination of three factors: the background, the lighting, and the disposition of the cables. The background is categorized as either *close*, when the cables rest on a surface (which constitutes the background of the image), or *far*, when the cables hang, revealing a background extending beyond them. Regarding lighting, two options exist: *concentrated* lighting, emitted by specific sources, like lamps, which create defined shadows and reflections; and *distributed* lighting, which provides ambient, diffuse light without a clear source, resulting in softer shadows and more diffused reflections. Both lighting conditions can be utilized to generate *close* background images, however, *far* background images only use *distributed* lighting to get the desired atmospheric effect. Finally, concerning the disposition of the cables, they can be either *aligned*, meaning that all of them are positioned closely together and have a similar direction, like the cables of a wiring harness; or *not-aligned* otherwise. This way, depending on the application (e.g., tangled cables in a box or wiring harnesses hanging), the user

can tailor the dataset by specifying the number of images of each category to be created.

The procedure followed to generate synthetic cable images from any of these categories is the same. The initial phase involves setting up and modeling a realistic environment in Blender. This scene incorporates various elements, namely the cables, floor, HDRI background, concentrated lights, wind, and camera. All these elements are described below.

Cables are modeled as deformable long and thin cylinders that can collide with each other. To achieve this, the mesh of a cable is initially defined as a long, narrow plane with multiple subdivisions, like a strip of cloth. Then, its deformable behavior is added with a cloth modifier. In this step, it is crucial to accurately adjust the cloth’s physical properties to achieve realistic cable shapes. After this, solidify and subdivision surface modifiers are added to give thickness to the cable. Finally, a collision modifier is added to allow collisions between cables.

Regarding the cable material, it is defined using procedural blocks, as can be seen in the bottom image of Fig. 3. The generated cables are monochromatic, however, to enhance realism, random dirt or imperfections are introduced. Both the cable and the dirt colors are defined using a Color Ramp block. Within this block, the position of the dirt color (usually dark colors) can be modified to adjust its density. As for the dirt pattern, it is defined with a Mapping and a Noise

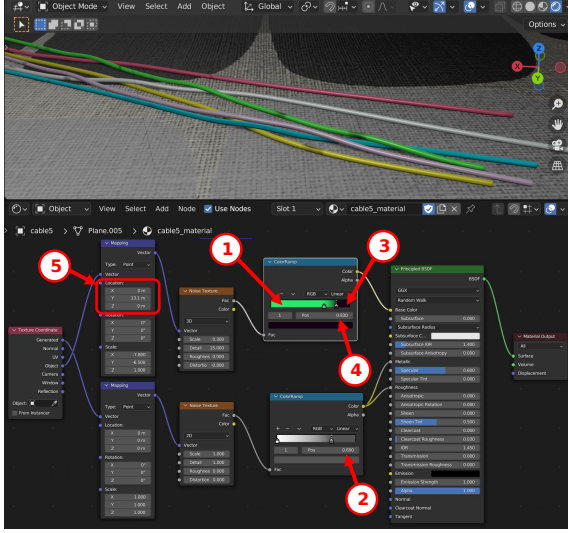


Fig. 3: Top: Cables with dirt/imperfections and reflections. Bottom: Procedural nodes to define the green cable material. The numbered items are the parameters subjected to random modifications: cable color (1), cable reflections (2), dirt/imperfections color (3), density (4) and pattern (5)

Texture block, connected to the Color Ramp Factor. Finally, the reflectivity of the cable is adjusted with its Metallic and Roughness properties, which are specified with another Color Ramp block. An example of the imperfections and reflections of the cables' material can be seen in the top image of Fig. 3.

The floor is the surface on which the cables are positioned when the camera image is rendered. Therefore, it is modeled as a passive rigid body plane with a collision modifier. Regarding its material, when the generated image has a *close* background, it is defined as a realistic texture, such as wood, tiles, or fabric, which can be downloaded from an asset catalog. On the other hand, when the image has a *far* background, the floor is hidden and disabled for the rendering, revealing the background extending beyond it (defined as an HDRI). In this case, the physical properties of the floor remain active, so the cables can still lie on top of it even if it is not visible.

The world is the 3D environment where the scene is located, and it defines the background and environment lighting. Depending on the image category, the world is defined either as a fixed

gray color, when the background is *close* and the lighting is *concentrated*, or as an HDRI otherwise. The selection of one or the other is done dynamically using a discrete Mix Shader block, which can be set to 0 or 1 to display the desired surface shader. An HDRI, which stands for a High Dynamic Range Image, is a spherical map image that provides detailed and comprehensive lighting, reflections, and environmental information in all directions. Therefore, these images form the backdrop of the *far* background images and provide the illumination for the *distributed* lighting images. Moreover, the brightness of the HDRI can be increased using two blocks: an Add Shader, which doubles the HDRI output color, and a Mix Shader that enables the gradual adjustment of brightness between the original HDRI and the amplified version.

Regarding the *concentrated* lighting, two types of light objects were employed: Point Light and Area Light. While both emit light from specific sources, Point Lights radiate light uniformly in all directions from a single point, whereas Area Lights emit light from a certain surface, primarily in a direction perpendicular to it. Finally, the last elements in the scene are the wind, which can move the cables and alter their shape, thus enriching the dataset; and the camera, positioned a certain distance above the floor and perpendicular to it, responsible for capturing and rendering the cables images.

Once the Blender scene is created, the second phase of the methodology consists of interacting with it to control the animation, perform variations on its elements, and acquire the images of the dataset. This process is controlled by a Python-based pipeline, which is depicted through a UML activity diagram in Fig. 4. First of all, the pipeline initializes the scene, loading all the assets (i.e., textures and HDRIs). Moreover, the loaded textures are used to create all the materials that can be assigned to the floor. After this, the pipeline checks the number of remaining images to be created from each category and selects one that hasn't been finished yet. Based on the selected category, the scene is modified to fit its requirements (e.g., the floor is hidden when the image has a *far* background) and random variations are applied to its elements, to generate a comprehensive dataset that covers a wide

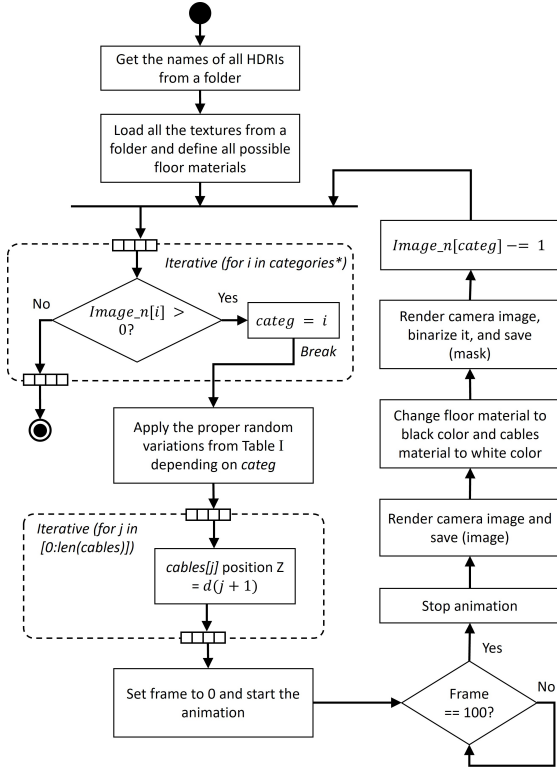


Fig. 4: UML activity diagram of the Synthetic cable generation process in Blender. * The categories specified in the diagram are the ones presented in Fig. 2

range of diverse scenarios and situations. All the considered variations are summarized in Table 1.

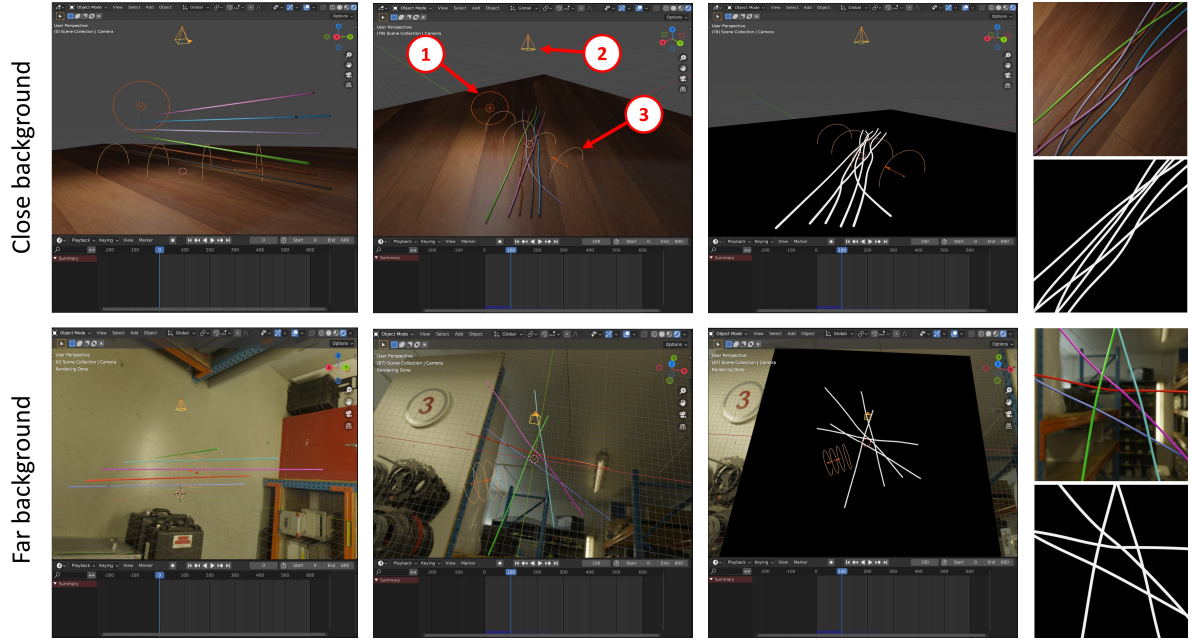
After this, the next step is to arrange the cables on the floor in different configurations to increase the realism of the images. However, to achieve realistic scenarios with multiple cables of various shapes jumbled together, it is not enough to spawn them in different locations over the floor, but their behavior must be simulated. Thus, the cables are initially positioned at different heights, separated by a distance d along the Z axis, as can be seen in Fig. 5(a), and then, the animation starts, letting them fall freely towards the floor. While they are falling, the wind (which has a random flow direction and power) can alter their position and shape. Upon reaching the floor, the cables collide with each other, leading to reactive motions that make them deform naturally, originating disorderly arrangements where they overlap each other. After a certain number of frames, these secondary motions cease, so the animation can be stopped

Table 1: Blender parameters subjected to random variations

Category	Varied parameter
Cables in scene	Number of cables: The excluded cables are moved out of the scene
	Cables size: Changing the camera distance
	Cables position (x,y):
	- Aligned: All cables move collectively a random distance, maintaining a slight random distance relative to each other. - Not aligned: Each cable moves individually a random distance.
Cables orientation (Rz):	- Aligned: Changes collectively a random angle for all the cables, maintaining a slight random relative angle to each other.
	- Not aligned: Changes a random angle for each cable individually.
Cables behavior	Cable deformability: Changing the vertex mass
Cables material (Check Fig. 3)	Cable color (RGB code)
	Cable material reflections: Metallic and roughness (colorRamp)
	Dirt/imperfections:
	- Color (RGB code, dark tones only), - Density (colorRamp), - Pattern (x, y displacement)
Wind	Direction: Changes in position (x,y,z) and orientation (Rx, Rz)
	Power: Strength and flow
Concentrated lighting	Type: Point light or area light
	Position (x,y,z) Power: Energy (both) and shadow_soft_size (point light)
Distributed lighting	Light emitted by an HDRI
	Brightness: The HDRI color is processed by an Add Shader and a variable Mix Shader block
Background close	Texture: 28 options from Poliigon [37]
	Pattern movement (x, y, Rz) Zoom/Scale
Background far	HDRI: 10 options from Poly Haven [38]
	HDRI rotation (Rz)

and the image can be captured (see Fig. 5(b)). Typically, these motions conclude by the frame 100, hence, this has been set as the final frame of the animation. It must be noted that the animation of the cables is the most time-consuming aspect of the dataset generation, so extending the number of frames would slow down the process.

Once the animation has been finalized, the camera proceeds to render the image of the cables and save it. For this step, the system can be



(a) Initial cable positions (frame 0) (b) Final configuration (frame 100) (c) Scene ready to render mask (d) Rendered images

Fig. 5: Animation of the synthetic cable segmentation database generation process. The numbered items show elements of the scene: point light (1), camera (2), wind (3)

configured to use either Eevee or Cycles as the rendering engine. Cycles will produce a higher-quality image, with more realistic illumination, shadows, and material reflections but at the expense of longer rendering times. Once the image is saved, it is necessary to generate its ground-truth segmentation mask. To do this, a plain black color material is assigned to the floor, and a plain white color material is assigned to all the cables, as can be seen in Fig. 5(c). Moreover, to avoid shadows and reflections on the cables, the emission strength of this new material is slightly increased. After these changes of material are made, a new image is rendered with the same camera. However, this image is not a valid binary segmentation mask yet as the captured colors are not purely black and white. Therefore, a minor post-processing is required to generate the mask as a binary image with a simple thresholding operation. Two examples of the output image and segmentation mask generated in this process can be seen in Fig. 5(d). Next, the scene restarts and the described process is repeated until all the images from all categories have been generated.

4 Synthetic segmentation dataset evaluation

To assess the quality of the synthetic cable images produced using the methodology described in the previous section, a dataset has been generated. This dataset was utilized to train six popular semantic segmentation models, which later were used to segment real cable images. The dataset comprises a total of 25000 synthetic images, divided into 5000 for each category, except for the two categories with *close* background and *distributed* lighting, each containing 2500 images. These images have a resolution of 512x512 pixels and contain from one to eight cables of eleven different sizes, ranging from 4 to 16 pixels per diameter. As for the backgrounds, 28 different textures from Poliigon [37] were used for the floor and 10 different HDRIs from Poly Haven [38] were used for the world. The entire dataset and all the assets used can be found in the project’s repository². The 25000 images and segmentation masks

²https://github.com/pablomalvido/cables_segmentation

of the dataset were generated using the Cycles render engine in 101 hours, i.e., an average of less than 15 seconds per image. The computer used for this was equipped with an Intel Core i7 CPU, 16 GB RAM, and an NVIDIA GeForce RTX 3070 GPU.

Six of the most popular DL semantic segmentation models were trained using this dataset: FCN [11], U-Net [16], LinkNet [17], FPN [18], PSPNet [19], and DeepLabV3+ [22]. The selection of these models was done after an extensive review which has been summarized in Section 2.1. The objective of this is not to compare the performance of the selected models but to validate the effectiveness of the synthetic dataset to train different kinds of models for later segmenting real cable images. Hence, three different backbone architectures were used: VGG16 [12], ResNet-50 [13], and MobileNetV2 [14]; all of them pre-trained on the ImageNet dataset. All the models were trained for 30 epochs using 20000 synthetic images for training and 5000 for validation.

These models were then used to segment 54 real cable images. Three different wiring harness models were used for these images. These wiring harnesses are composed of six, ten, and eleven cables of different colors, whose diameters range between 1.34mm and 2.1mm. As a result, the taken images present multiple difficulties for the segmentation of the cables, such as constant overlapping, occlusions, entanglements, adjacent cables, different colors, and small diameters. Moreover, different lighting conditions and multiple *close* and *far* backgrounds were used, many of them representative of industrial and robotic setups, which is the main target application. Due to all this, the testing dataset is considered to cover a great variety of complex scenarios and segmentation problems and to be representative of its potential applications.

To assess the segmentation of these 54 real images with the models trained on the synthetic dataset, it was necessary to manually label them. To do this an interactive algorithm was developed to determine the cables skeleton. The algorithm allows the user to click a certain number of points along the cable skeleton and then it approximates it with a polynomial function. The order of this function is automatically determined by cross-validation, considering orders from 1 to 8. After the estimation of the skeleton curve, the user can accept or reject it, depending on how well it

Table 2: Performance of popular segmentation models trained with the generated synthetic cables images. The IoU and Dice coefficient values are expressed in percentages

Architecture	Backbone	Synthetic images		Real images	
		IoU	Dice	IoU	Dice
FCN	VGG16	84.96	91.66	72.69	83.94
DeepLabV3+	ResNet-50	91.00	95.16	76.10	86.21
PSPNet	ResNet-50	89.38	95.12	77.64	87.20
FPN	ResNet-50	94.47	97.01	74.71	85.11
LinkNet	MobileNetV2	96.27	98.09	70.96	81.88
U-Net	MobileNetV2	91.59	95.55	78.11	87.59

adjusts to the cable shape. The number of points required to characterize each cable will depend on its shape. After this, the skeleton curve widens to fill the width of the cable. This labeling process took a total of 6.5 hours of active work, which means an average of around 7 minutes per image, depending on the number of cables per image. If this is compared with the proposed synthetic image generation method, it can be seen that only considering the labeling time, the manual method is almost 30 times more time-consuming. This disparity becomes even bigger when considering the time of other concomitant tasks, such as scene/cables preparation and image acquisition to ensure the diversity of the dataset. Moreover, the synthetic image generation method is not just much faster but also fully automatic, requiring zero manual work.

Table 2 summarizes the results of the six segmentation models for the 5000 synthetic validation cable images and the 54 real cable images. In both cases, the models were trained with the synthetic dataset. Two metrics were used to evaluate the performance of the models, the IoU (Intersection over Union) and the Dice coefficient. The obtained results show excellent performance for both the synthetic and the real images for all the models. As expected, the segmentation of the synthetic images is slightly better, obtaining almost perfect results (IoU over 84% and Dice over 90% for all of them). Nevertheless, the models exhibited remarkable performance on real image segmentation as well (IoU surpassing 70% and Dice over 80% in all cases), especially when considering the complexity of the images. This proves the effectiveness of the proposed methodology for creating realistic synthetic cable datasets

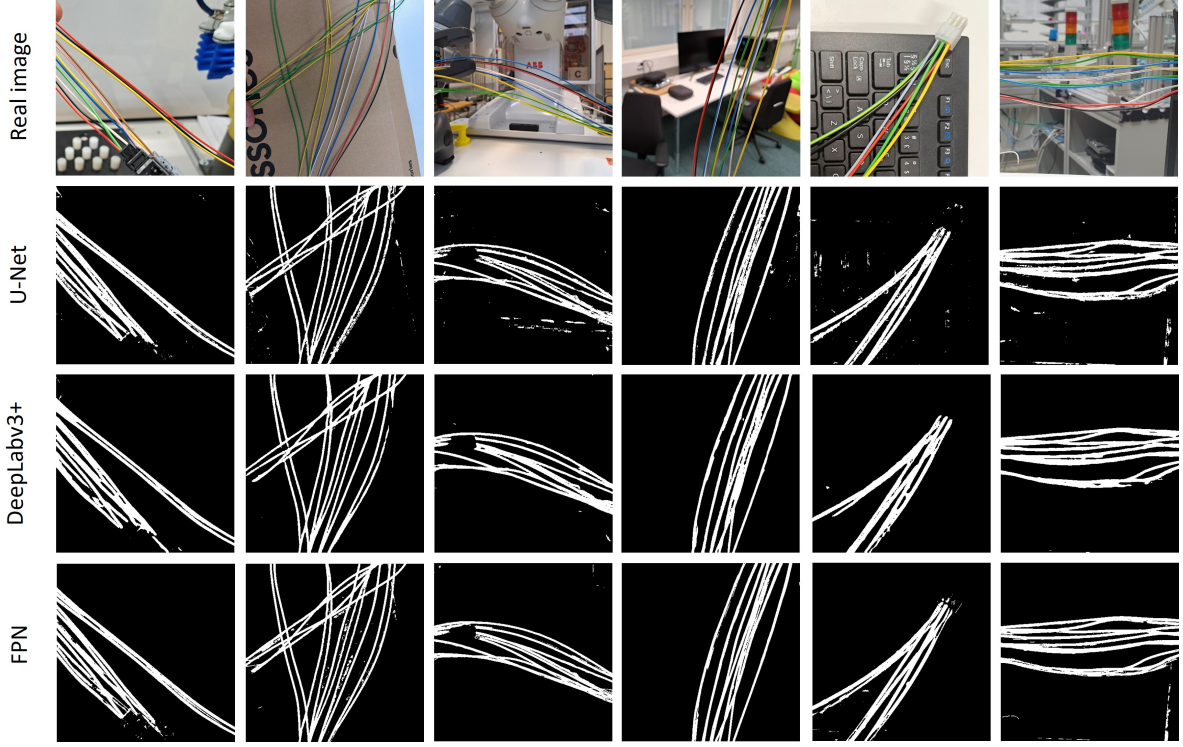


Fig. 6: Segmentation of real wiring harness images with popular DL segmentation models trained with the generated synthetic cables images

for training DL segmentation models. Additionally, Fig. 6 showcases the masks predicted by three of the evaluated models for certain real cable images. The complete set of results is available in the project’s repository, which also contains the weights of trained models and a couple of algorithms for testing the models with new images.

5 Conclusions and future work

This paper presents a novel methodology to automatically generate realistic synthetic cable datasets to train DL segmentation models. The methodology leverages the power of Blender to create photo-realistic scenes including multiple cables and different backgrounds. To achieve this level of realism, special attention has been paid to the definition of the lighting and the cables material. The scene and all its elements are controlled and altered by a Python pipeline, which automates the generation of diverse cable images and their corresponding masks. This process includes

the animation of the scene during a fixed number of frames to let the cables fall from a certain height and deform naturally when colliding between them after landing on the floor. Moreover, to guarantee the diversity of the dataset, the pipeline applies random variations on the scene before starting the generation of each image. These variations include, among others, changes in the cables (e.g., number, size, position, deformability, and material properties), the background, and the lighting of the scene.

To evaluate the effectiveness of the proposed methodology, it has been utilized to generate a dataset of 25000 synthetic cable images, and this has been used to train six popular semantic segmentation models. These models were then used to segment 54 real wiring harness images, which included multiple cables of multiple colors and sizes, with entanglements, and occlusions; multiple backgrounds; and varying light conditions. The six models obtained a great performance, with an IoU surpassing 70% and a Dice coefficient over 80% for all of them, proving the realism of

the synthetic dataset and its efficacy in training segmentation models. Additionally, a comparison between the time required to generate an image segmentation dataset manually versus employing the proposed methodology revealed that the latter not only eradicated the time spent in active work but was also approximately 30 times faster.

In future works, the methodology will be extended to generate synthetic datasets for different purposes, such as instance segmentation or object recognition. Additionally, its potential to generate synthetic datasets for other kinds of deformable objects will be explored, broadening its scope to not only encompass objects deformable in one dimension but also in two and three, such as clothes or certain food products.

Acknowledgements. This work was supported by the European Commission’s Horizon 2020 Framework Programme through Project REMODEL - Robotic technologies for the manipulation of complex deformable linear objects - under Grant Agreement 870133.

Declarations

Competing Interests The authors declare that they have no conflict of interest.

References

- [1] Sanchez, J., Corrales, J.-A., Bouzgarrou, B.-C., Mezouar, Y.: Robotic manipulation and sensing of deformable objects in domestic and industrial applications: a survey. *The International Journal of Robotics Research* **37**(7), 688–716 (2018) <https://doi.org/10.1177/0278364918779698>
- [2] Lv, N., Liu, J., Jia, Y.: Dynamic Modeling and Control of Deformable Linear Objects for Single-Arm and Dual-Arm Robot Manipulations. *IEEE Transactions on Robotics* **38**(4), 2341–2353 (2022) <https://doi.org/10.1109/TRO.2021.3139838>
- [3] Pirozzi, S., Natale, C.: Tactile-based manipulation of wires for switchgear assembly. *IEEE/ASME Transactions on Mechatronics* **23**(6), 2650–2661 (2018)
- [4] Kicki, P., Bednarek, M., Lembicz, P., Mierzwiak, G., Szymko, A., Kraft, M., Walas, K.: Tell Me, What Do You See?—Interpretable Classification of Wiring Harness Branches with Deep Neural Networks. *Sensors* **21**(13), 4327 (2021) <https://doi.org/10.3390/s21134327>
- [5] Caporali, A., Galassi, K., Zanella, R., Palli, G.: FASTDLO: Fast Deformable Linear Objects Instance Segmentation. *IEEE Robotics and Automation Letters* **7**(4), 9075–9082 (2022) <https://doi.org/10.1109/LRA.2022.3189791>
- [6] Ortiz, A., Antich, J., Oliver, G.: A particle filter-based approach for tracking undersea narrow telecommunication cables. *Machine Vision and Applications* **22**(2), 283–302 (2011) <https://doi.org/10.1007/s00138-009-0199-6>
- [7] Malvido Fresnillo, P., Vasudevan, S., Mohammed, W.M., Martinez Lastra, J.L., Perez Garcia, J.A.: An approach based on machine vision for the identification and shape estimation of deformable linear objects. *Mechatronics* **96**, 103085 (2023) <https://doi.org/10.1016/j.mechatronics.2023.103085>
- [8] Pal, N.R., Pal, S.K.: A review on image segmentation techniques. *Pattern Recognition* **26**(9), 1277–1294 (1993) [https://doi.org/10.1016/0031-3203\(93\)90135-J](https://doi.org/10.1016/0031-3203(93)90135-J)
- [9] Minaee, S., Boykov, Y., Porikli, F., Plaza, A., Kehtarnavaz, N., Terzopoulos, D.: Image Segmentation Using Deep Learning: A Survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **44**(7), 3523–3542 (2022) <https://doi.org/10.1109/TPAMI.2021.3059968>
- [10] Alzubaidi, L., Zhang, J., Humaidi, A.J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M.A., Al-Amidie, M., Farhan, L.: Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. *Journal of Big Data* **8**(1), 53 (2021) <https://doi.org/10.1186/s40537-021-00444-8>

- [11] Long, J., Shelhamer, E., Darrell, T.: Fully convolutional networks for semantic segmentation. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (2015)
- [12] Simonyan, K., Zisserman, A.: Very Deep Convolutional Networks for Large-Scale Image Recognition. In: 3rd International Conference on Learning Representations (ICLR2015) (2015). <https://doi.org/10.48550/arXiv.1409.1556> . arXiv:1409.1556
- [13] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (2016)
- [14] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.-C.: Mobilenetv2: Inverted residuals and linear bottlenecks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (2018)
- [15] Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A.C., Fei-Fei, L.: ImageNet Large Scale Visual Recognition Challenge. International Journal of Computer Vision **115**(3), 211–252 (2015) <https://doi.org/10.1007/s11263-015-0816-y>
- [16] Ronneberger, O., Fischer, P., Brox, T.: U-Net: Convolutional Networks for Biomedical Image Segmentation. In: Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015, pp. 234–241 (2015). https://doi.org/10.1007/978-3-319-24574-4_28
- [17] Chaurasia, A., Culurciello, E.: LinkNet: Exploiting encoder representations for efficient semantic segmentation. In: 2017 IEEE Visual Communications and Image Processing (VCIP), pp. 1–4 (2017). <https://doi.org/10.1109/VCIP.2017.8305148>
- [18] Lin, T.-Y., Dollar, P., Girshick, R., He, K., Hariharan, B., Belongie, S.: Feature pyramid networks for object detection. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (2017)
- [19] Zhao, H., Shi, J., Qi, X., Wang, X., Jia, J.: Pyramid scene parsing network. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (2017)
- [20] Chen, L.-C., Papandreou, G., Kokkinos, I., Murphy, K., Yuille, A.L.: DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs. IEEE Transactions on Pattern Analysis and Machine Intelligence **40**(4), 834–848 (2018) <https://doi.org/10.1109/TPAMI.2017.2699184>
- [21] Zhao, R., Xie, M., Feng, X., Guo, M., Su, X., Zhang, P.: Interaction semantic segmentation network via progressive supervised learning. Machine Vision and Applications **35**(2), 1–14 (2024) <https://doi.org/10.1007/s00138-023-01500-4>
- [22] Chen, L.-C., Zhu, Y., Papandreou, G., Schroff, F., Adam, H.: Encoder-decoder with atrous separable convolution for semantic image segmentation. In: Proceedings of the European Conference on Computer Vision (ECCV) (2018)
- [23] Yarram, S., Yuan, J., Yang, M.: Adversarial structured prediction for domain-adaptive semantic segmentation. Machine Vision and Applications **33**(5), 1–13 (2022) <https://doi.org/10.1007/s00138-022-01308-8>
- [24] Shorten, C., Khoshgoftaar, T.M.: A survey on Image Data Augmentation for Deep Learning. Journal of Big Data **6**(1), 60 (2019) <https://doi.org/10.1186/s40537-019-0197-0>
- [25] Zanella, R., Caporali, A., Tadaka, K., De Gregorio, D., Palli, G.: Auto-generated Wires Dataset for Semantic Segmentation with Domain-Independence. In: 2021 International Conference on Computer, Control and Robotics (ICCCR), pp. 08–10. IEEE. <https://doi.org/10.1109/ICCCR49711.2021.9349395>

- [26] Wahd, A.S., Kim, D., Lee, S.-I.: Cable Instance Segmentation with Synthetic Data Generation. In: 2022 22nd International Conference on Control, Automation and Systems (ICCAS), pp. 1533–1538. IEEE. <https://doi.org/10.23919/ICCAS55662.2022.10003680>
- [27] Zhou, S., Bi, Y., Wei, X., Liu, J., Ye, Z., Li, F., Du, Y.: Automated detection and classification of spilled loads on freeways based on improved YOLO network. *Machine Vision and Applications* **32**(2), 1–12 (2021) <https://doi.org/10.1007/s00138-021-01171-z>
- [28] Madaan, R., Maturana, D., Scherer, S.: Wire detection using synthetic data and dilated convolutional networks for unmanned aerial vehicles. In: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 24–28. <https://doi.org/10.1109/IROS.2017.8206190>
- [29] Pharr, M., Humphreys, G.: *Physically Based Rendering, Second Edition: From Theory To Implementation*, 2nd edn. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (2010)
- [30] Moiola, G.: *Introduction to Blender 3.0: Learn Organic and Architectural Modeling, Lighting, Materials, Painting, Rendering, and Compositing with Blender*. Apress, Berkeley, CA (2022). <https://doi.org/10.1007/978-1-4842-7954-0>
- [31] Denninger, M., Sundermeyer, M., Winkelbauer, D., Zidan, Y., Olefir, D., Elbadrawy, M., Lodhi, A., Katam, H.: *BlenderProc*. arXiv (2019) <https://doi.org/10.48550/arXiv.1911.01911>
- [32] Adam, R., Janciauskas, P., Ebel, T., Adam, J.: Synthetic Training Data Generation and Domain Randomization for Object Detection in the Formula Student Driverless Framework. In: 2022 International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICEC-CME), pp. 16–18. IEEE. <https://doi.org/10.1109/ICECCME55909.2022.9987772>
- [33] Caporali, A., Pantano, M., Janisch, L., Regulin, D., Palli, G., Lee, D.: A Weakly Supervised Semi-Automatic Image Labeling Approach for Deformable Linear Objects. *IEEE Rob. Autom. Lett.* **8**(2), 1013–1020 (2023) <https://doi.org/10.1109/LRA.2023.3234799>
- [34] Barth, R., IJsselmuiden, J., Hemming, J., Henten, E.J.V.: Data synthesis methods for semantic segmentation in agriculture: A Capsicum annum dataset. *Computers and Electronics in Agriculture* **144**, 284–296 (2018) <https://doi.org/10.1016/j.compag.2017.12.001>
- [35] Barth, R., Hemming, J., Van Henten, E.J.: Optimising realism of synthetic images using cycle generative adversarial networks for improved part segmentation. *Comput. Electron. Agric.* **173**, 105378 (2020) <https://doi.org/10.1016/j.compag.2020.105378>
- [36] Qiu, W., Yuille, A.: UnrealCV: Connecting Computer Vision to Unreal Engine. In: *Computer Vision – ECCV 2016 Workshops*, pp. 909–916. Springer, Cham, Switzerland (2016). https://doi.org/10.1007/978-3-319-49409-8_75
- [37] Textures. <https://www.poliigon.com/textures> Accessed 2023-12-29
- [38] HDRIs • Poly Haven. <https://polyhaven.com/hdris/> Accessed 2023-12-29